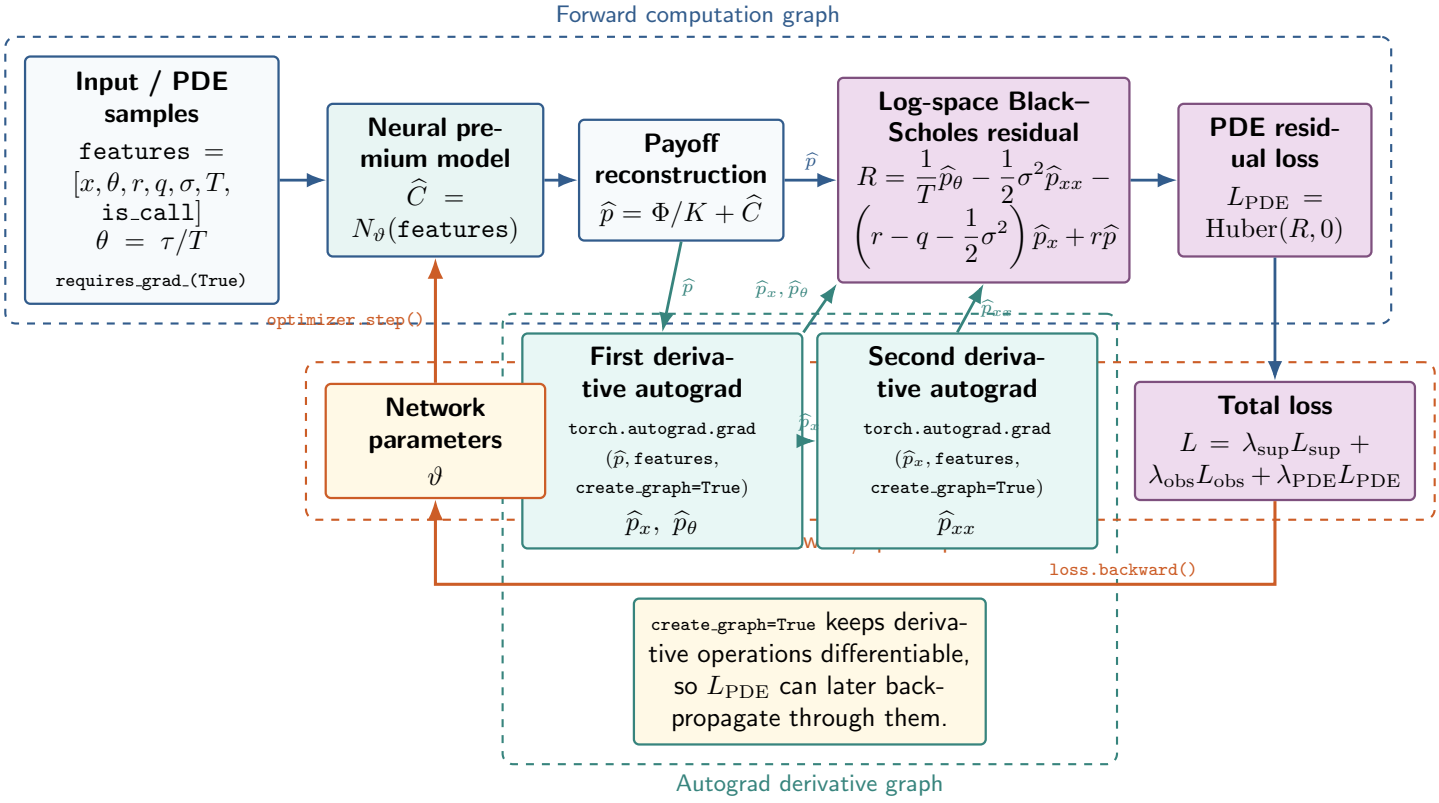

Forward computation graph
Input / PDE samples
features = [x, θ, r, q, σ, T, is_call]
θ = τ/T
requires_grad_(True)
Neural premium model
Ĉ = N_ϑ(features)
Payoff reconstruction
p̂ = Φ/K + Ĉ
p̂
Log-space Black–Scholes residual
R = (1/T) p̂_θ − (1/2)σ² p̂_xx − (r − q − (1/2)σ²) p̂_x + r p̂
PDE residual loss
L_PDE = Huber(R, 0)
optimizer.step()
p̂
p̂_x, p̂_θ
p̂_xx
Network parameters
ϑ
First derivative autograd
torch.autograd.grad
(p̂, features, create_graph=True)
p̂_x, p̂_θ
Second derivative autograd
torch.autograd.grad
(p̂_x, features, create_graph=True)
p̂_xx
Total loss
L = λ_sup L_sup + λ_obs L_obs + λ_PDE L_PDE
loss.backward()
create_graph=True keeps derivative operations differentiable, so L_PDE can later backpropagate through them.
Autograd derivative graph